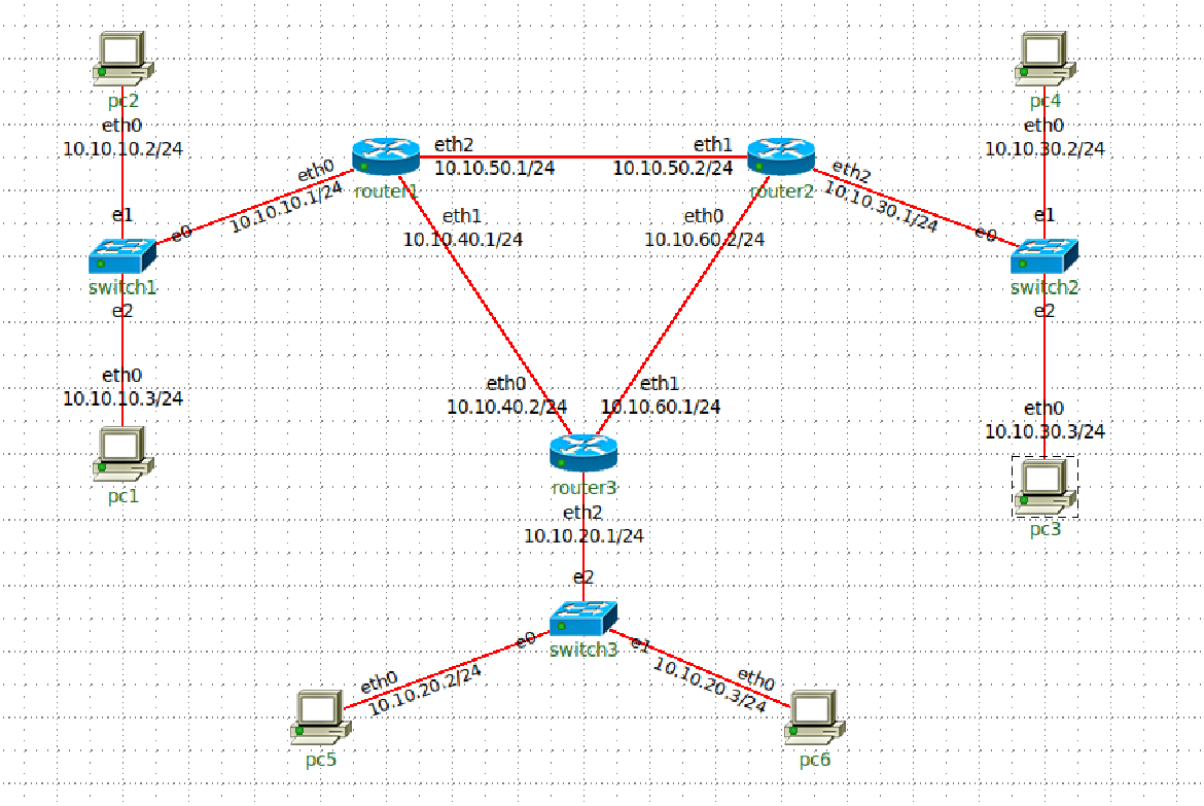
KOMMRE – Pitanja i odgovori -> 2. Lab

3. Protokoli mrežnog sloja

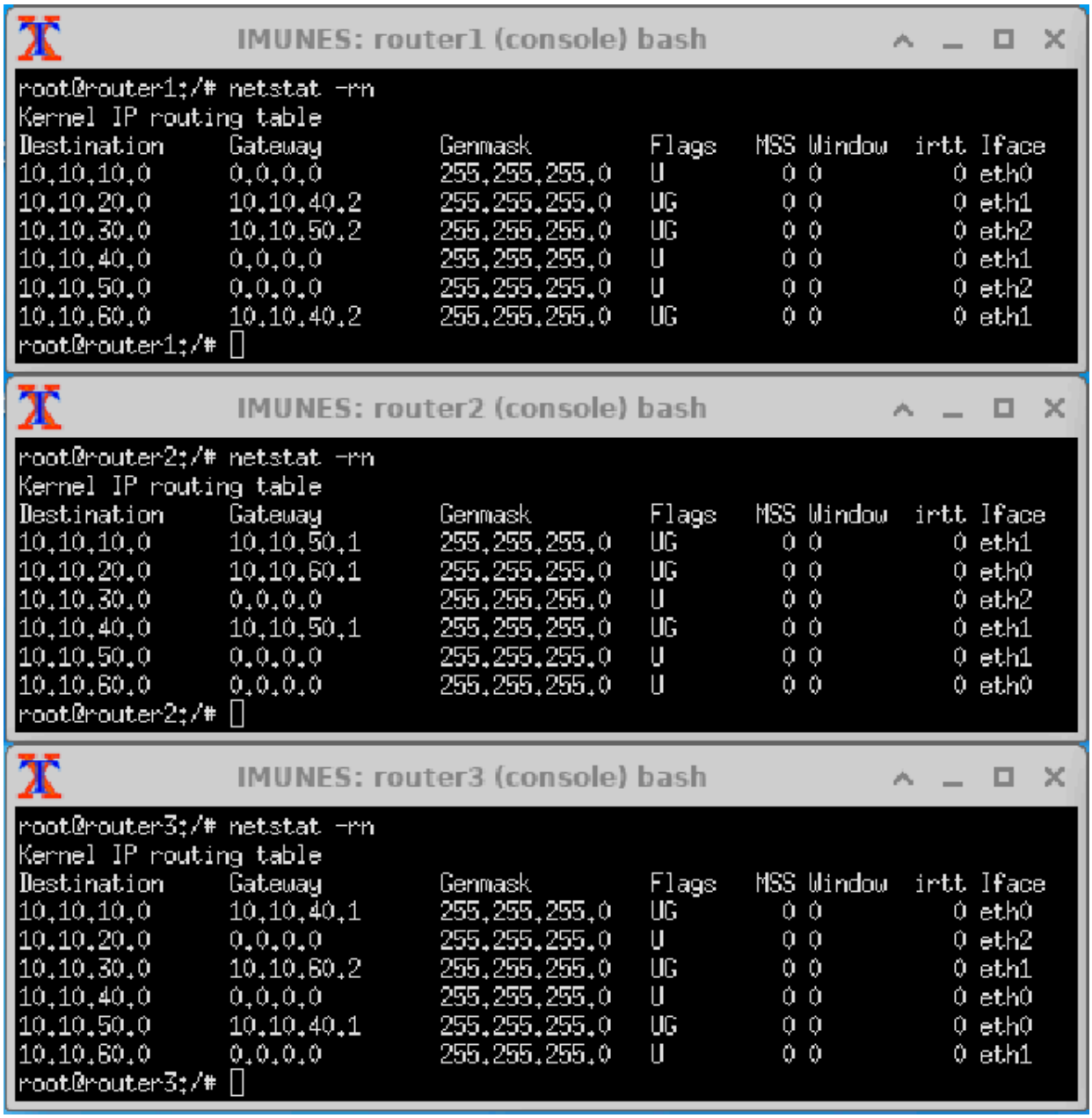
Zadatak 22. (Vlastito izrađena topologija) U emulatoru/simulatoru IMUNES konstruirajte mrežu koja sadrži tri podmreže povezane usmjeriteljima (Kao na slici). Konfigurirajte statičko usmjeravanje između svih podmreža. Sučeljima računala i usmjeritelja dodijelite IP-adrese iz raspona 10.10.10.0/24, 10.10.20.0/24, 10.10.30.0/24, 10.10.40.0/24, 10.10.50.0/24 i 10.10.60.0/24. Ispišite tablice usmjeravanja svih računala i usmjeritelja.



(Topologija s podmrežama kao u udžbeniku)

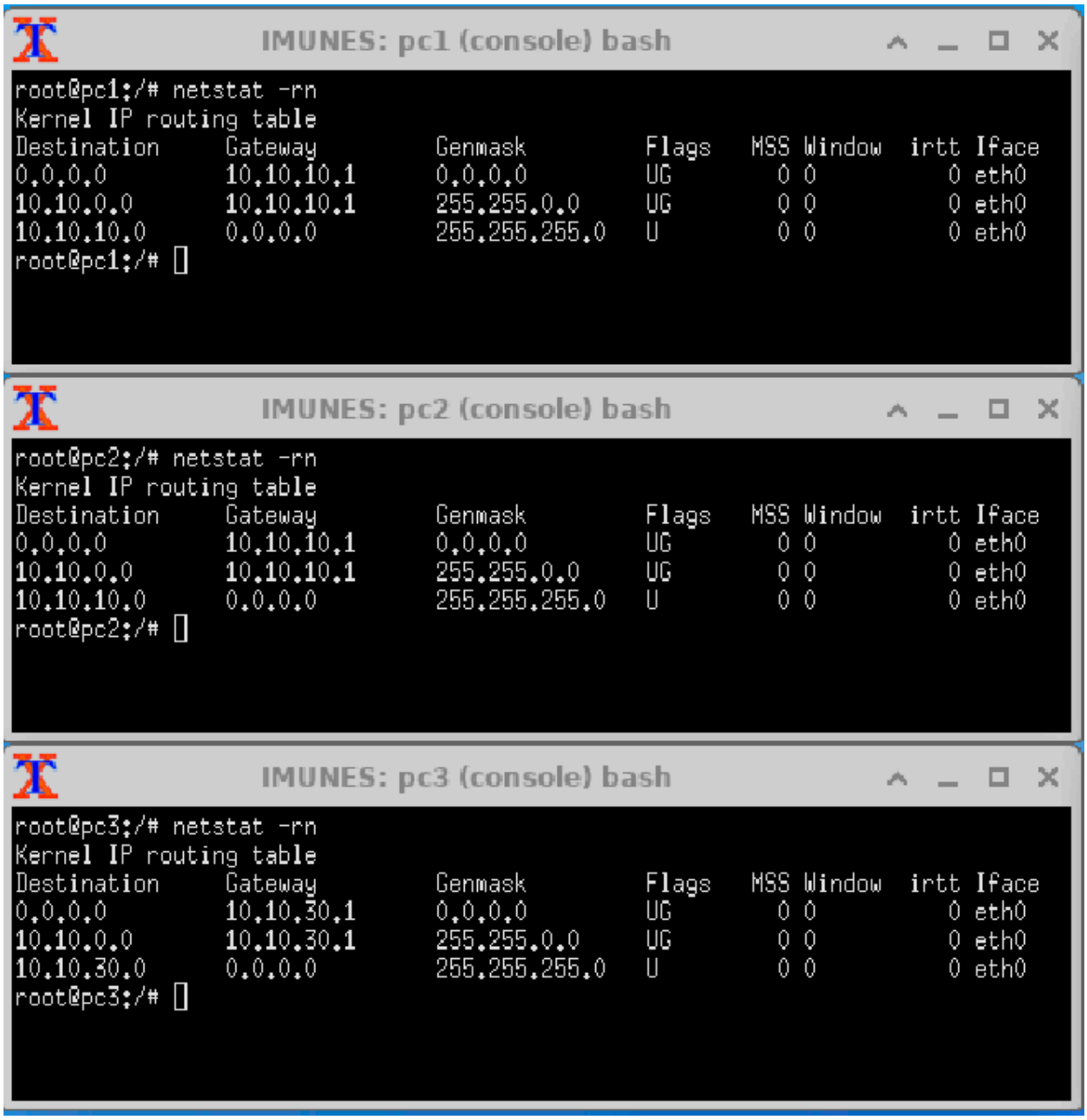
Tablice usmjeravanja su dobivene korištenjem komande netstat -rn u kojoj vidimo redom Destination, Gateway (Next Stop), Mask, par informacija nebitnih za ovaj zadatak i na kraju iface za interfaceove.

Tablice usmjeravanja router-a:

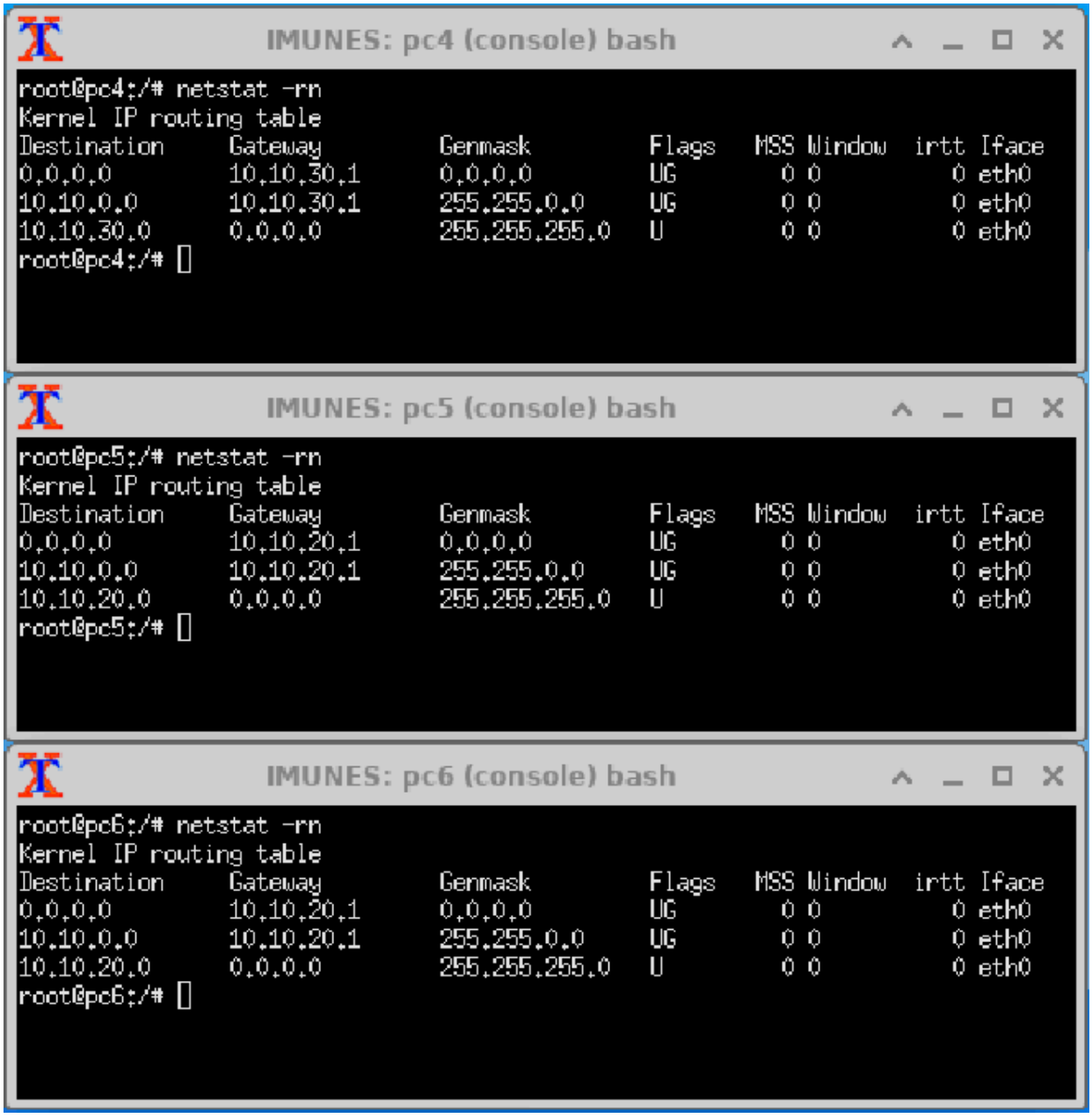


(router1 – router 3)

Tablice usmjeravanja pc-eva:



(pc1 – pc3)


IMUNES: pc4 (console) bash
root@pc4:/# netstat -rn
Kernel IP routing table
Destination Gateway Genmask Flags MSS Window irtt Iface
0.0.0.0 10.10.30.1 0.0.0.0 UG 0 0 0 eth0
10.10.0.0 10.10.30.1 255.255.0.0 UG 0 0 0 eth0
10.10.30.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
root@pc4:/#
IMUNES: pc5 (console) bash
root@pc5:/# netstat -rn
Kernel IP routing table
Destination Gateway Genmask Flags MSS Window irtt Iface
0.0.0.0 10.10.20.1 0.0.0.0 UG 0 0 0 eth0
10.10.0.0 10.10.20.1 255.255.0.0 UG 0 0 0 eth0
10.10.20.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
root@pc5:/#
IMUNES: pc6 (console) bash
root@pc6:/# netstat -rn
Kernel IP routing table
Destination Gateway Genmask Flags MSS Window irtt Iface
0.0.0.0 10.10.20.1 0.0.0.0 UG 0 0 0 eth0
10.10.0.0 10.10.20.1 255.255.0.0 UG 0 0 0 eth0
10.10.20.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
root@pc6:/#

(pc4-pc6)

Zadatak 23. (Vlastito izrađena topologija) U emulatoru/simulatoru IMUNES konstruirajte mrežu koja sadrži tri podmreže povezane usmjeriteljima (Slika kao i na 22.). Konfigurirajte statičko usmjeravanje tako da dođe do petlje u usmjeravanju. Sučeljima računala i usmjeritelja dodijelite IP-adrese iz raspona 10.10.10.0/24, 10.10.20.0/24, 10.10.30.0/24, 10.10.40.0/24, 10.10.50.0/24 i 10.10.60.0/24. Ispišite tablice usmjeravanja svih računala i usmjeritelja. Pomoću alata Wireshark utvrdite što se tada događa s paketima koji "uđu" u petlju te komentirajte svoja zapažanja.

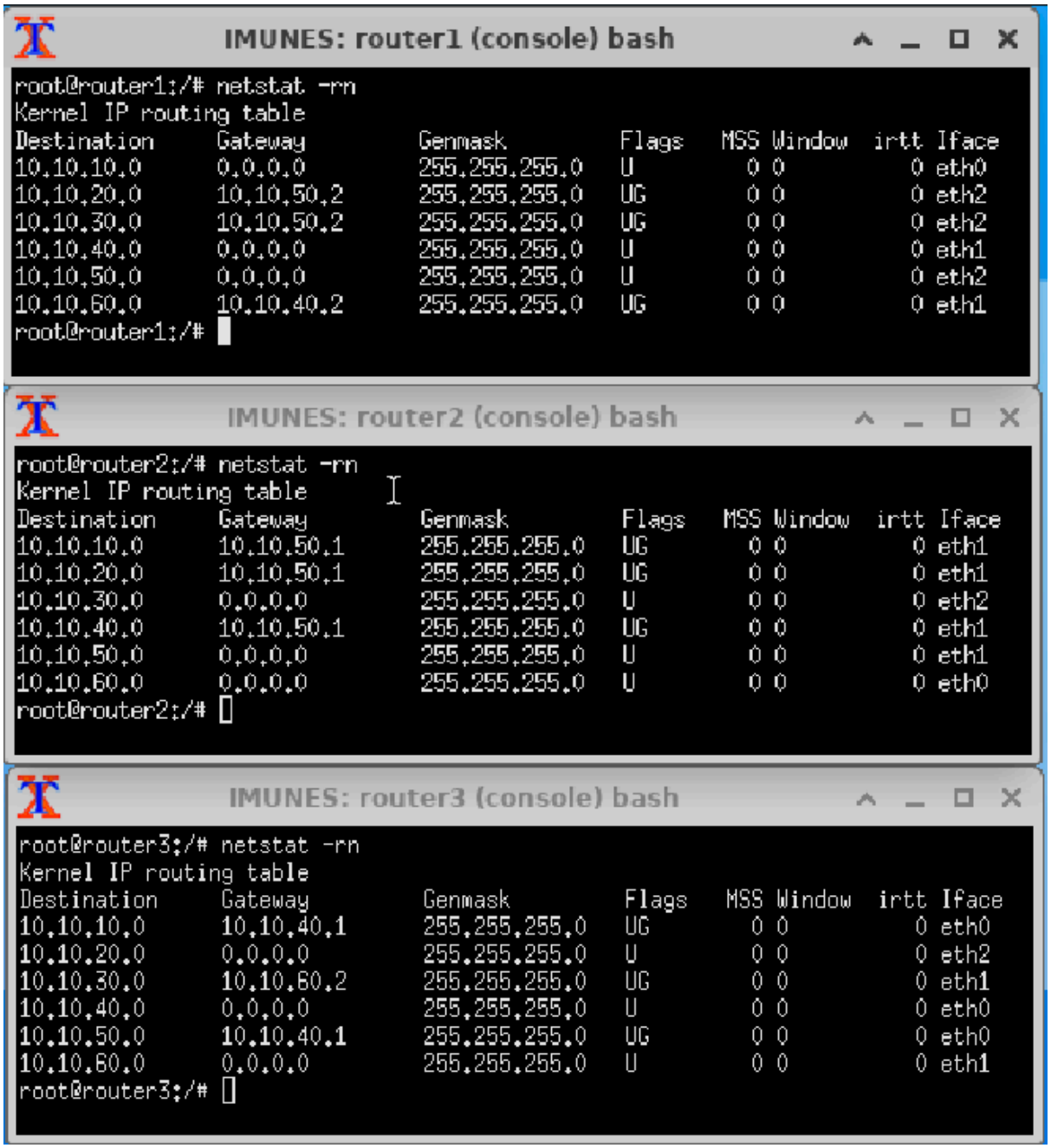
Cilj: Poslati ping paket i dobiti nazad „Time to live exceeded“ jer time znamo da je paket ušao u petlju. Ping naredba je dana s ping *adresa* -c 1 tako da bi se samo jedan request slao.

Postoje dvije stvari koje smo mogli napraviti da dobijemo neku vrstu „petlje:“

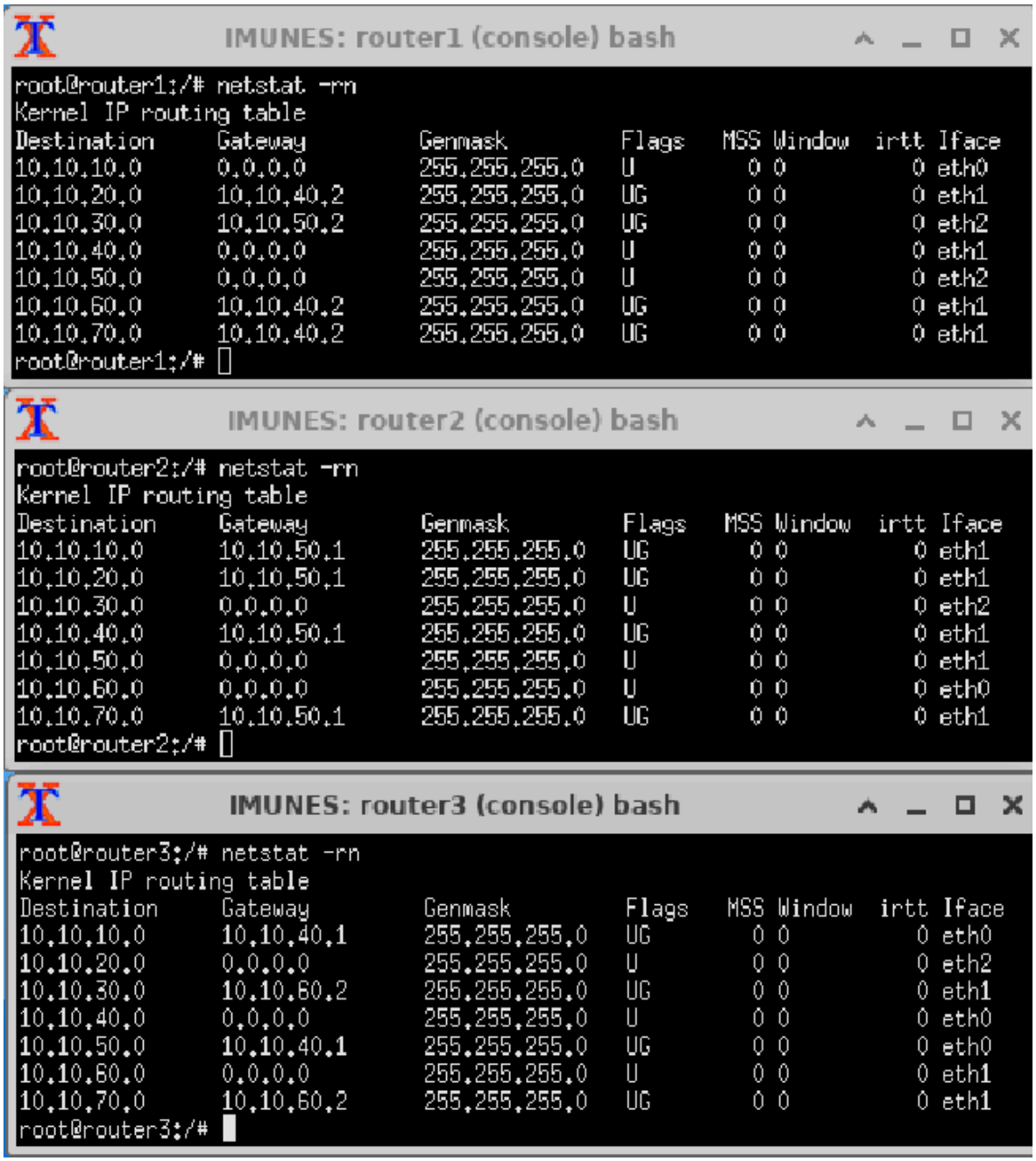
1. Napraviti ping-pong efekt između dva rutera koji su na kontra strani od adrese u pingu (npr ako šaljemo s 10.10.10.2 na 10.10.20.2 onda napraviti ping-pong routera 1 i routera 2 tako da stavimo u tablice na mjesta 10.10.20.0/24 destinaciju 10.10.50.2 ili 10.10.50.1 ovisno o routeru.
2. Poslati ping na adresu koja ne postoji u topologiji (u mojem primjeru 10.10.70.0/24) i onda gledati na nekom od routera kako se vrti u krug (po meni prava petlja).

U oba slučaja tablice za pc1-pc6 ostaju nepromijenjene.

Za slučaj 1 tablice usmjeravanja routera izgledaju ovako:



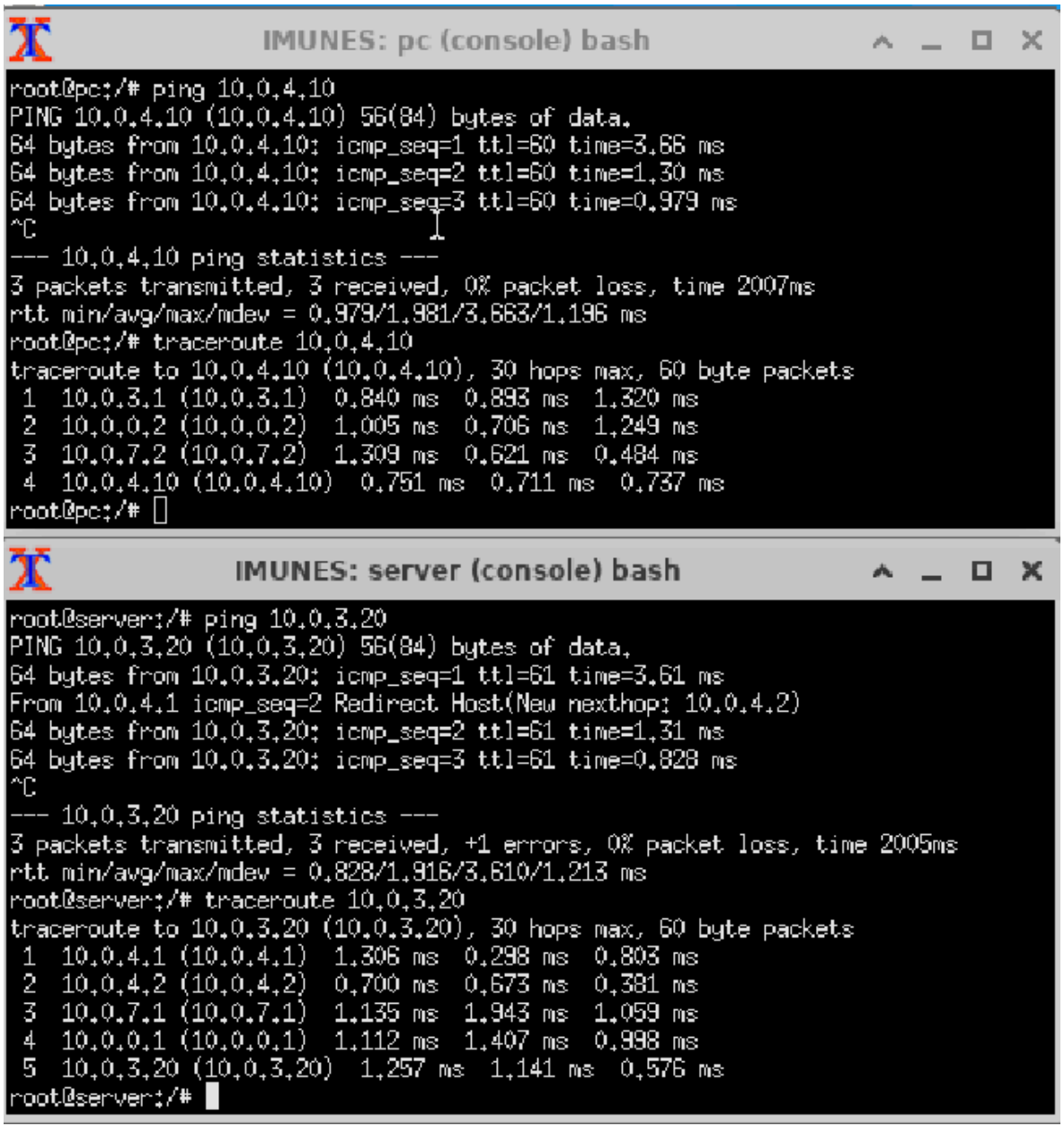
Za slučaj 2 tablice usmjeravanja routera izgledaju ovako:



U oba slučaja čekamo da TTL na pingu dođe do 0 kako bi došlo do „Time to live exceeded.“

Zadatak 24. (Topologija RIP/RIP1.imn) Započnite simulaciju i s računala pc provjerite dostupnost (naredba ping) poslužitelja server i očitajte TTL vrijednost iz ispisa. Zatim, s poslužitelja server provjerite dostupnost računala pc (naredba ping) i očitajte tu TTL vrijednost iz ispisa. Kojim putem idu paketi u jednom, a kojim putem u drugom slučaju? Ukratko objasnite zašto se međusobno razlikuju?

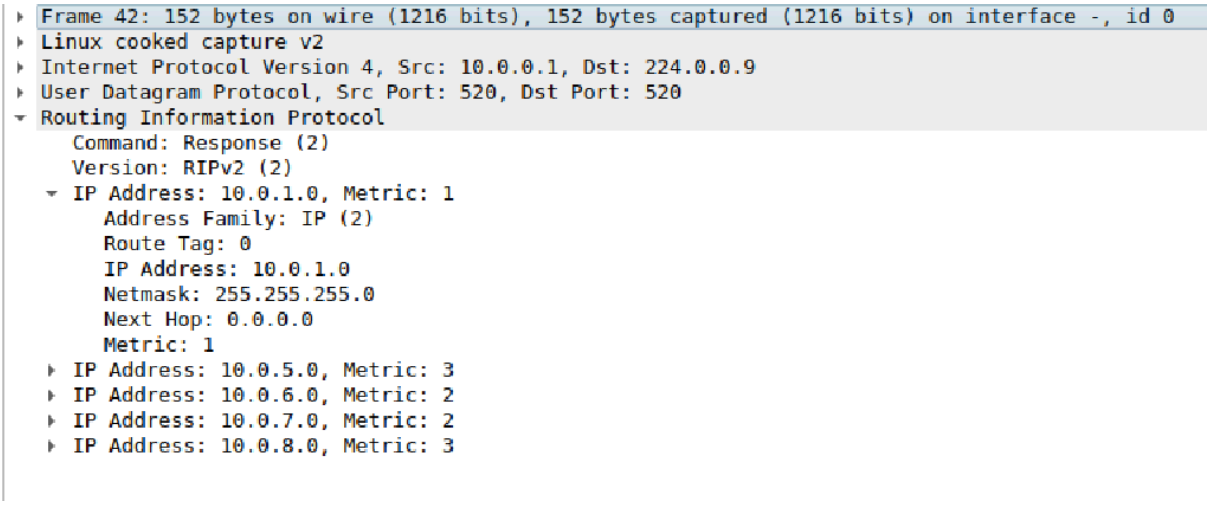
Paketi u smjeru PC -> Server idu: Router 1 -> Router 2 -> Router 7 -> Server (TTL na pingu je 60)
A u smjeru Server -> PC: Router 6 -> Router 7 -> Router 2 -> Router 1 -> PC (TTL na pingu 61)
Do razlike dolazi zbog razlike u tablicama usmjeravanja ovisno o smjeru i udaljenosti, što očito vidimo iz Router 6 pa Router 7 na putu od Servera do PCa. Zanimljivo je da na kraju put od servera do pca ima manje hopova zbog većeg ttla :)



Zadatak 25. (Topologija RIP/RIP.imn) U emulatoru/simulatoru IMUNES, pomoću alata Wireshark snimite paket koji pripada protokolu RIP, proučite njegov sadržaj, te ga ukratko komentirajte.

Protokol RIP (Routing Information Protocol) u sebi sadrži osnovne informacije Command (opis što se šalje) i Version (koja verzija RIPa se koristi) nakon koje ide set od 20 okteta metrike vezane uz određenu IP adresu koja sadrži sljedeće:

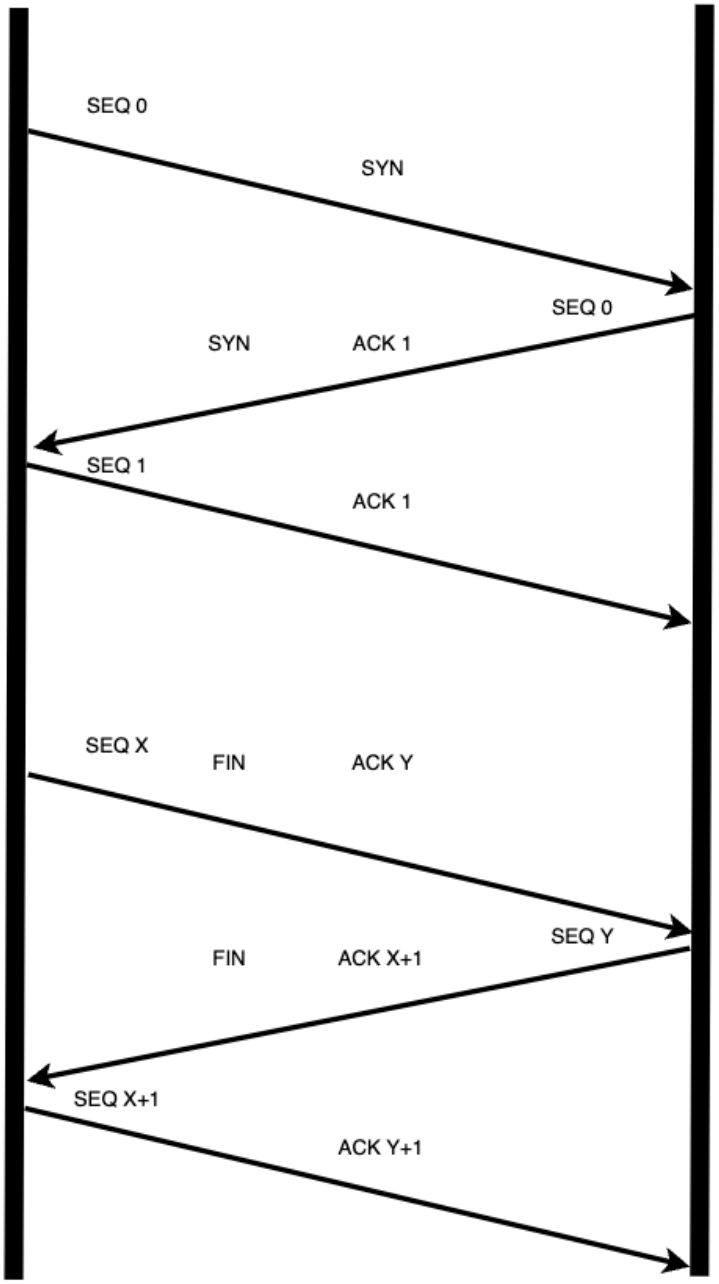
- Vrstu adrese: u ovom slučaju 32 bitna IP adresa
- Oznaku Rute: za razlikovanje ruta RIPa i drugih protokola
- IP adresu: adresa podmreže za koju se određuje ruta
- Netmask: maska podmreže
- Next Hop: sljedeći skok



4. Protokoli transportnog sloja

Zadatak 28. Učitajte u IMUNES mrežu Ping/ping.imn. Pomoću alata Wireshark snimite proizvoljan TCP-promet koji pripada jednoj vezi te odredite segmente koji se razmjenjuju u fazama uspostave veze i raskida veze (za generiranje TCP-prometa iskoristite alat netcat).

a. Skicirajte razmjenu segmenata za te dvije faze, uz navođenje korištenih TCP-zastavica.



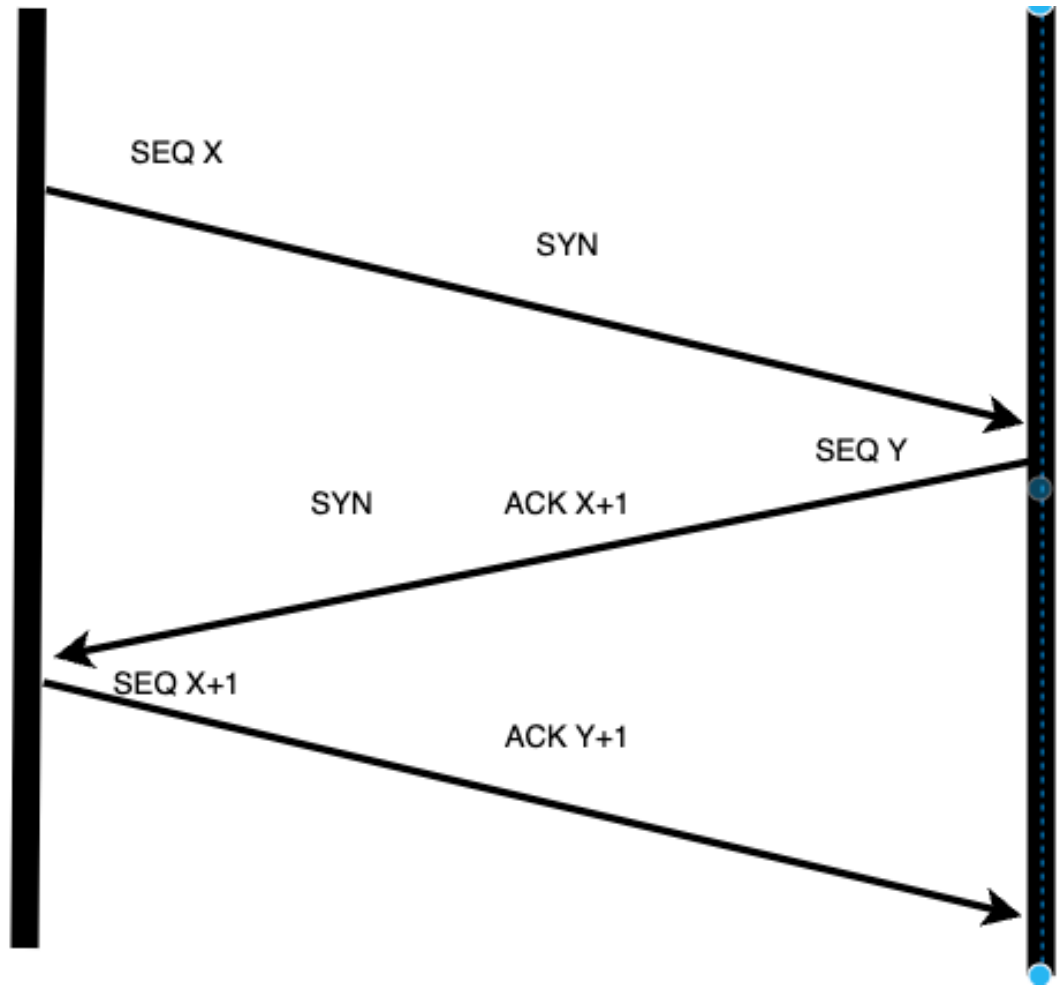
b. Za uhvaćeni promet, odredite koje se adrese i vrata koriste na izvorištu i odredištu. Imaju li svi segmenti istu četvorku {izvorišna IP-adresa, izvorišna vrata, odredišna IPadresa, odredišna vrata}?

Uvijek sadrže istu četvorku, u mojem navedenom primjeru su:

Pošiljatelj: PC1 → 10.0.0.21:33586

Primatelj: Server → 10.0.8.10:150

c. Skicirajte razmjenu nekoliko TCP-segmenata u fazi trajanja veze, uz navođenje korištenih TCP-zastavica.

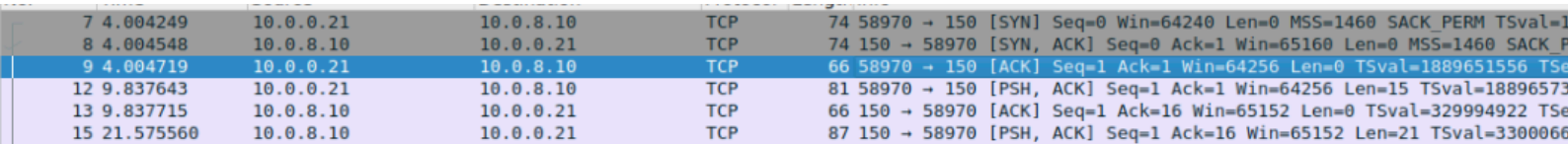


d. Utvrdite na koji se način koriste potvrde u TCP-vezi. Komentirajte.

U TCP-vezi, potvrde (dalje ACKovi) koriste se kumulativno. Točnije, svaki ACK će potvrditi sve oktete do tog rednog broja (danog u njegovom polju). ACK efektivno označava koji oktet treba biti sljedeći.

e. Snimite proizvoljan TCP-promet (koji pripada jednoj vezi) i utvrdite veličine prozora.

Kada šaljemo SYN requestove veličine prozora su velike i iznose 64240 za pošiljatelja i 65160 za primatelja. Nakon sinkronizacije dižu se na 64256 za pošiljatelja i 65152 za primatelja na kojemu ostaju do kraja veze. To nije stalno jer je na drugom pokretanju gdje su se odbijali paketi bilo spušteno na 502 i 510 okteta koji su onda ostali do kraja veze.



7	4.004249	10.0.0.21	10.0.8.10	TCP	74 58970 → 150 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1
8	4.004548	10.0.8.10	10.0.0.21	TCP	74 150 → 58970 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_P
9	4.004719	10.0.0.21	10.0.8.10	TCP	66 58970 → 150 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1889651556 TSe
12	9.837643	10.0.0.21	10.0.8.10	TCP	81 58970 → 150 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=15 TSval=18896573
13	9.837715	10.0.8.10	10.0.0.21	TCP	66 150 → 58970 [ACK] Seq=1 Ack=16 Win=65152 Len=0 TSval=329994922 TSe
15	21.575560	10.0.8.10	10.0.0.21	TCP	87 150 → 58970 [PSH, ACK] Seq=1 Ack=16 Win=65152 Len=21 TSval=3300066
16	21.576044	10.0.0.21	10.0.8.10	TCP	66 58970 → 150 [ACK] Seq=16 Ack=22 Win=64256 Len=0 TSval=1889669127 T
18	35.322512	10.0.0.21	10.0.8.10	TCP	101 58970 → 150 [PSH, ACK] Seq=16 Ack=22 Win=64256 Len=35 TSval=188968
19	35.322585	10.0.8.10	10.0.0.21	TCP	66 150 → 58970 [ACK] Seq=22 Ack=51 Win=65152 Len=0 TSval=330020407 TS
21	47.367732	10.0.8.10	10.0.0.21	TCP	91 150 → 58970 [PSH, ACK] Seq=22 Ack=51 Win=65152 Len=25 TSval=330032
22	47.367847	10.0.0.21	10.0.8.10	TCP	66 58970 → 150 [ACK] Seq=51 Ack=47 Win=64256 Len=0 TSval=1889694919 T
25	59.919219	10.0.0.21	10.0.8.10	TCP	94 58970 → 150 [PSH, ACK] Seq=51 Ack=47 Win=64256 Len=28 TSval=188970
26	59.919282	10.0.8.10	10.0.0.21	TCP	66 150 → 58970 [ACK] Seq=47 Ack=79 Win=65152 Len=0 TSval=330045003 TS
27	66.340504	10.0.8.10	10.0.0.21	TCP	76 150 → 58970 [PSH, ACK] Seq=47 Ack=79 Win=65152 Len=10 TSval=330051
28	66.340940	10.0.0.21	10.0.8.10	TCP	66 58970 → 150 [ACK] Seq=79 Ack=57 Win=64256 Len=0 TSval=1889713892 T
30	68.296827	10.0.8.10	10.0.0.21	TCP	66 150 → 58970 [FIN, ACK] Seq=57 Ack=79 Win=65152 Len=0 TSval=3300533
31	68.337388	10.0.0.21	10.0.8.10	TCP	66 58970 → 150 [ACK] Seq=79 Ack=58 Win=64256 Len=0 TSval=1889715889 T
34	70.439296	10.0.0.21	10.0.8.10	TCP	66 58970 → 150 [FIN, ACK] Seq=79 Ack=58 Win=64256 Len=0 TSval=188971
35	70.439519	10.0.8.10	10.0.0.21	TCP	66 150 → 58970 [ACK] Seq=58 Ack=80 Win=65152 Len=0 TSval=330055524 TS

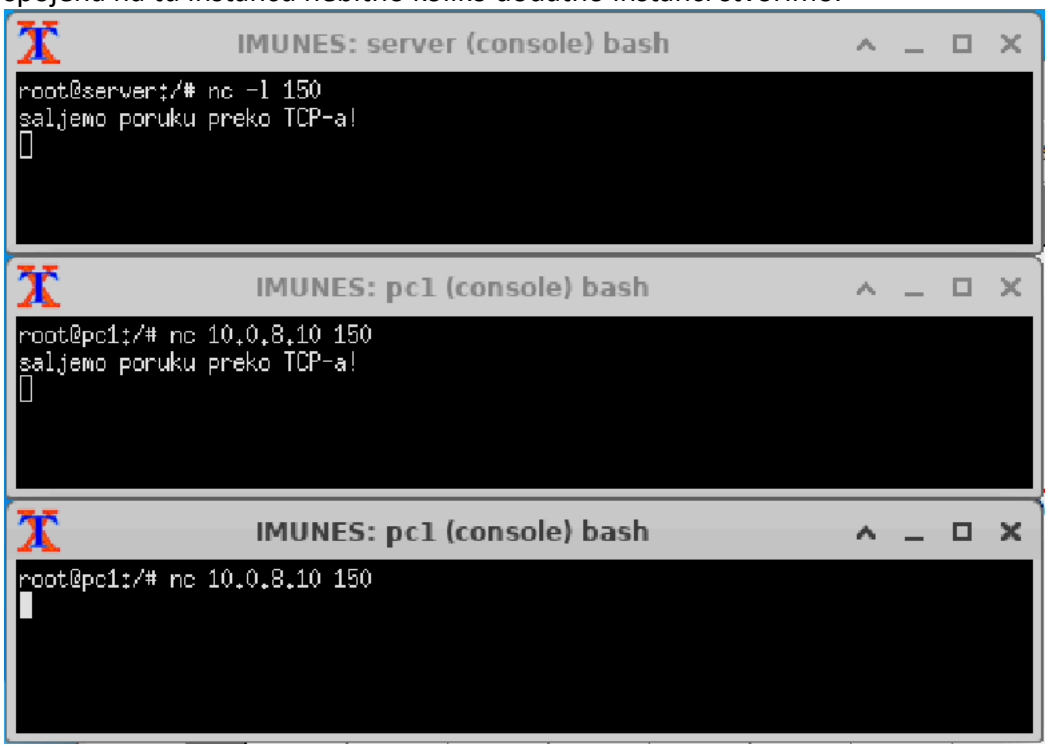
Mijenja li se veličina prozora često u tijeku trajanja TCP-veze? Objasnite.

Kratki odgovor: ovisi.

Duži odgovor: U nekakvim stabilnim vezama gdje oba kraja veze nisu spora dosta brzo će se dići window s 1 MSS na neki stabilni broj nakon čega će tamo i ostati. Veličine prozora će se mijenjati u nestabilnijim uvjetima gdje postoji veća šansa da paket ne dođe i/ili ako je jedan od krajeva veze spor jer će se onda njegov prozor brzo popuniti. U takvim uvjetima veličina prozora će se više mijenjati ovisno o mogućoj maksimalnoj kvaliteti konekcije u danim uvjetima.

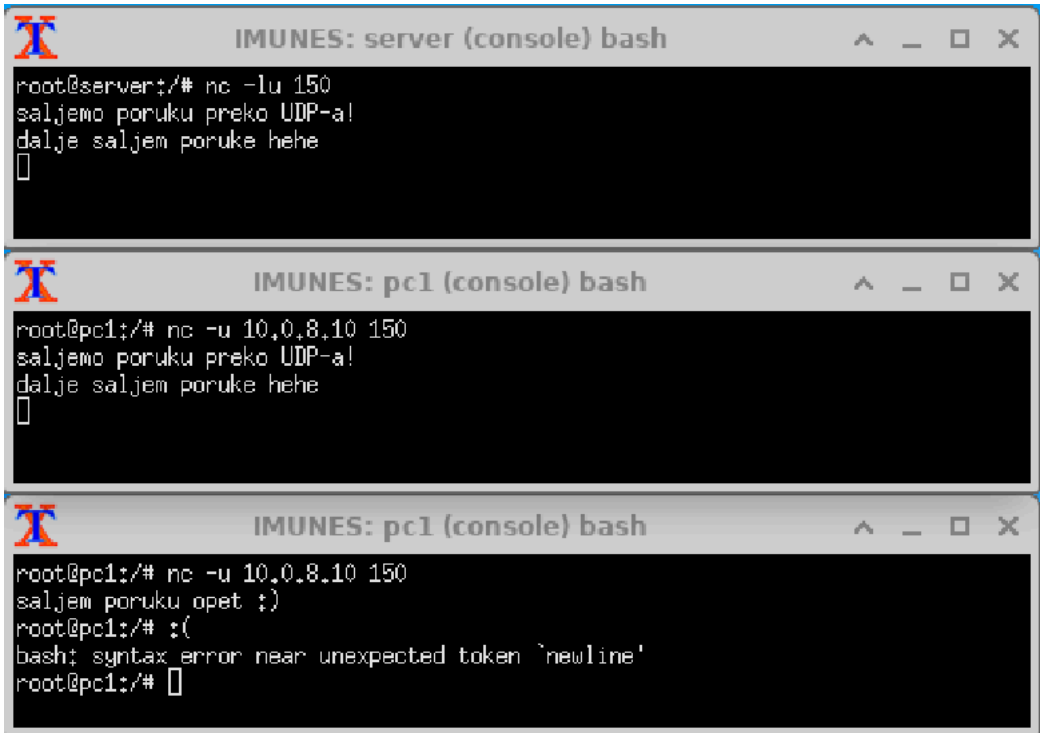
Zadatak 29. (Topologija Ping/ping.imn) Utvrdite mogu li se na jednom računalu pokrenuti dva procesa koji slušaju na istim vratima. Komentirajte.

Na jednome računalu možemo pokrenuti više instanci slušanja na istim vratima, ali će samo jedan od njih zapravo primati TCP prijenos.
To ima smisla jer je TCP spojna usluga koja će od kada se prva instanca spoji nastaviti biti spojena na tu instancu nebitno koliko dodatno instanci stvorimo.



Zadatak 30. (Topologija Ping/ping.imn) Ukoliko se alatu netcat ne zada protokol koji će koristiti, on podrazumijeva protokol TCP. Ponovite pokus iz prethodnog zadatka uz korištenje protokola UDP te komentirajte.

Preko UDP protokola dolazi do uspostavljanja veze tek s prvim poslanim podacima, jedino ovdje nakon uspostavljanja konekcije dolazi do zatvaranja drugih instanci s istog PC1 kada oni pošalju poruku.



Zadatak 31. (Topologija Ping/ping.imn) Primijetite da, iako su bitno različiti po svojstvima, protokoli UDP i TCP po funkcionalnosti spadaju u transportni sloj referentnog modela OSI. Objasnite zašto.

Transportni sloj kao takav zahtjeva end-to-end prijenos podataka unutar mreže s određenom kvalitetom usluge. TCP i UDP, kolikogod bili različiti, odrađuju taj posao prijenosa (transporta) podataka unutar mreže ovisno o tome što se od njih očekuje.
TCP je napravljen kako bi zadovoljio prijenos podataka gdje gubitak djela informacija ili dovođenje informacija u različitom poretku postaje kobno za paket. Samo zamislite koliko bi loše izgledalo da mail koji pošaljete dođe naopačke… TCP prioritizira tu spojnost i red u konekciji nad čistom brzinom, iako s svim metodama kontrole to postaje poprilično impresivno.
UDP ima totalno drugi zadatak, prijenos što više podataka što brže bez obzira na redoslijed ili točnost. Na mjestima gdje se UDP koristi nije bitan redoslijed jer je važnije imati brzu i koherentnu konekciju nad točnim redoslijedom i kvalitetom poslanih podataka. Na YouTube videu nije bitno ako je jedan od 60 frameova u sekundi kriv ili na krivom mjestu, ali je bitno da je u 128 kbps zvuka tog videa dobro preneseno preko 70 kbps (na primjer, znam da podaci nisu skroz točni) jer to daje bolju kvalitetu usluge aplikacijskog sloja koji se oslanja na transportni sloj.

Zadatak 32. (Topologija Ping/ping.imn) Pokušajte prouzročiti gubitak TCP-segmenata. Na koji način možete utvrditi da je došlo do gubitaka? Možete li izazvati gubitke paketa bez mijenjanja karakteristika poveznica mreže?

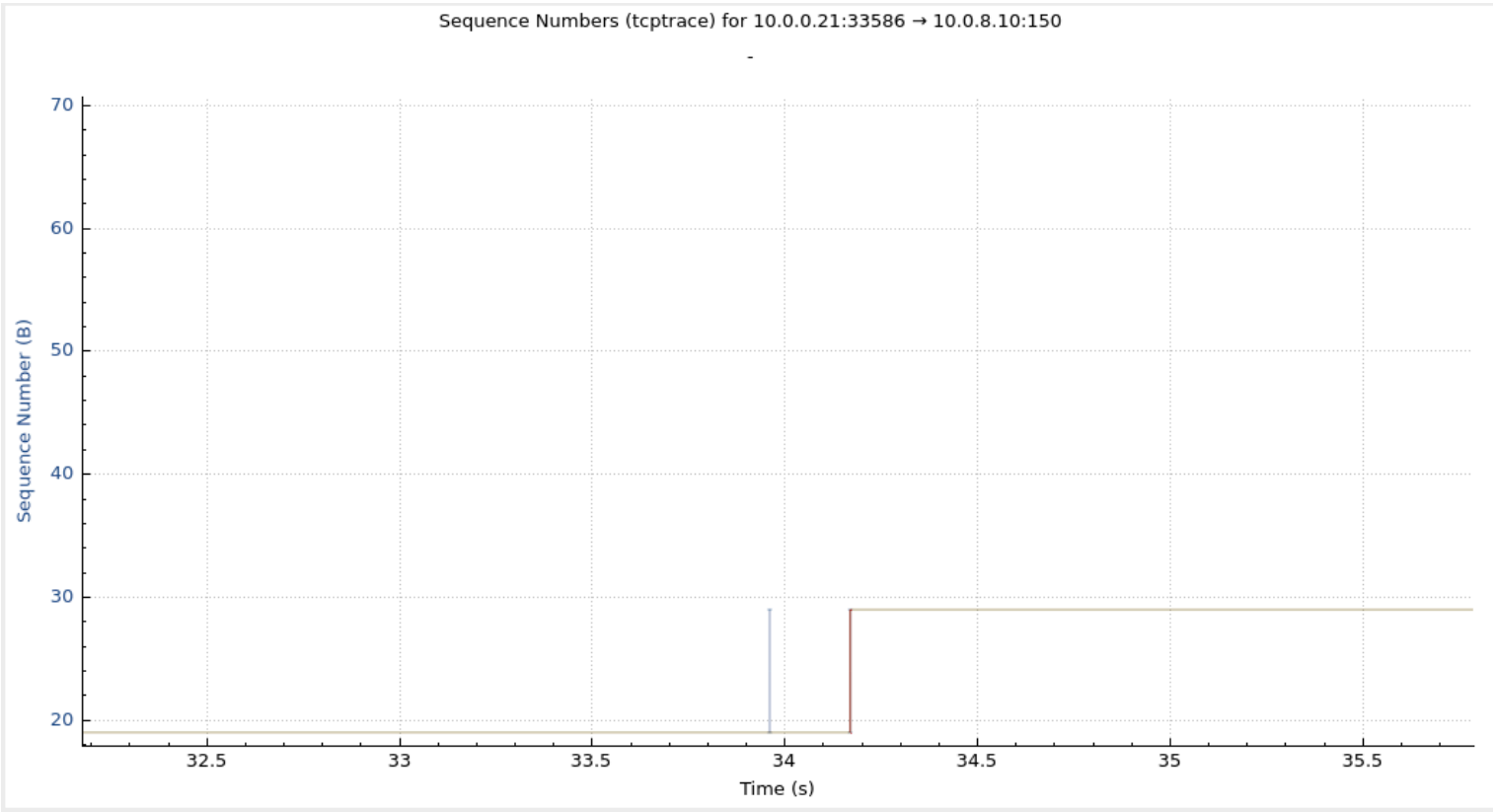
Malo odgovaram unazad ali dobro. Bez mijenjanja karakteristika poveznica mreže teško je prouzročiti gubitak TCP-segmenata. U slučaju gdje su mreža i njezine karakteristike poveznica loše početno konfigurirane može doći do gubitaka segmenata, ali opet u rubnim slučajevima (npr. Visok latency ili sporo vraćanje ACKova, pretrpavanje ili gušenje mreže, loša konekcija i slično).

U ovom zadatku modificirao sam mrežu na način da sam dodao pravilo na Server (10.0.8.10) gdje će odbaciti 50% paketa koji mu stignu (točna komanda: tc qdisc add dev eth0 root netem loss 50%, našao na internetu) tako da se sigurno morala desiti retransmisija zbog ne primanja ACKa.

Sada na početni dio pitanja. Možemo utvrditi da je došlo do gubitaka ako prođe RTO bez primanja ACKa od druge strane u konekciji ili ako primimo 3 DUPACKa za redom. Inače to vidimo ako se pošalje paket s istim Seq i Ack poljem kao i paket prije njega od strane pošiljatelja. Wireshark dodatno to olakšava time što pored paketa koji su ponovno slani stavlja „TCP Retransmission flag“ koji govori zašto je došlo do retransmisije.

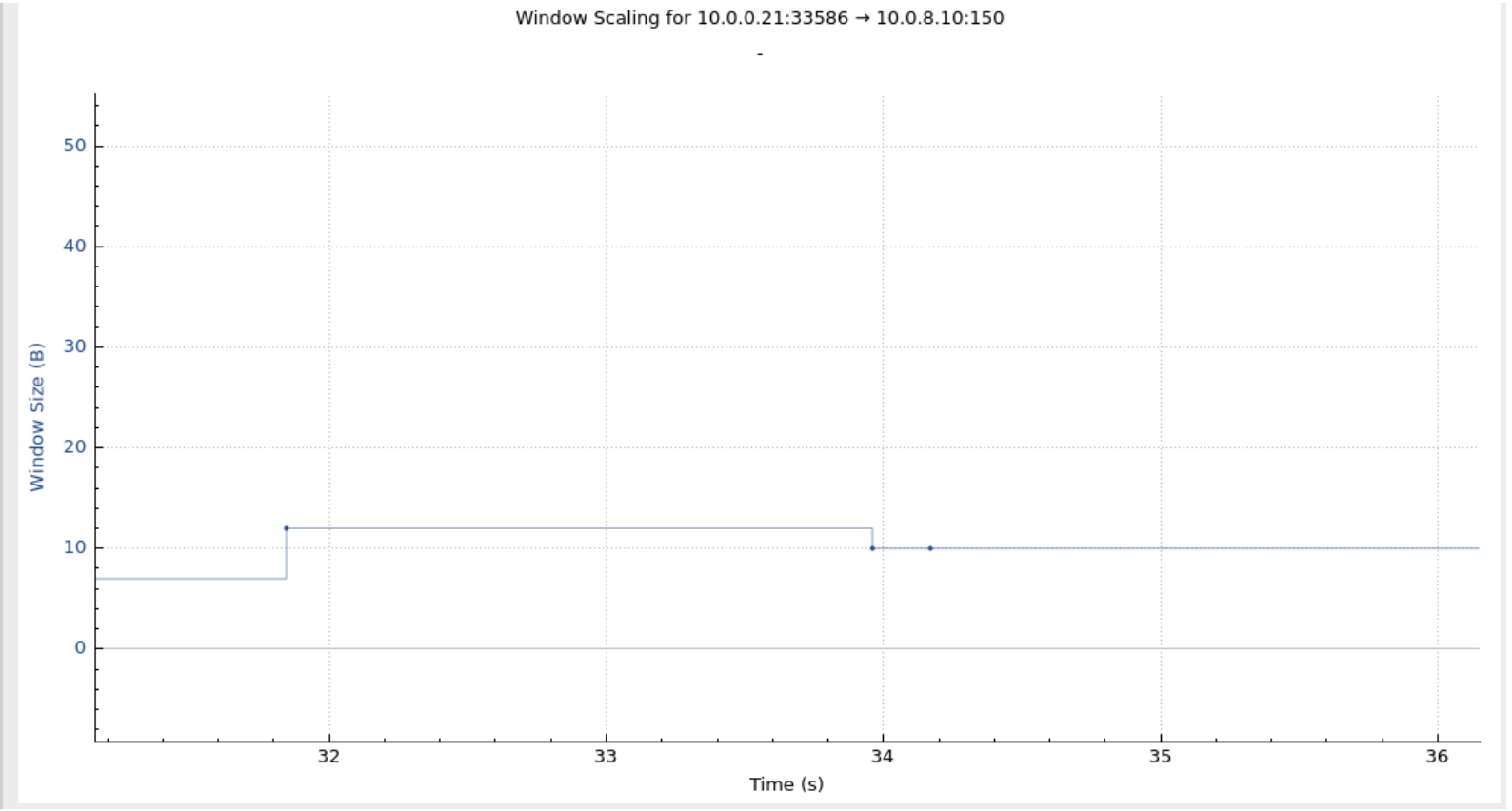
30	34.169379	10.0.0.21	10.0.8.10	TCP	76 [TCP Retransmission] 33586 → 150 [PSH, ACK] Seq=20 Ack=103
31	34.169433	10.0.8.10	10.0.0.21	TCP	78 150 → 33586 [ACK] Seq=103 Ack=30 Win=510 Len=0 TSval=16562

U Wiresharku također možemo na grafu „Time sequence“ vidjeti kako se ponovno šalju paketi koji ne dobiju svoj ack.



Zadatak 33. (Topologija Ping/ping.imn) Pokušajte identificirati promet koji pripada jednoj TCP-vezi za vrijeme u kojem dolazi do gubitka segmenata. Utvrdite što se tada događa s potvrdama i veličinom prozora.

Kada dođe do gubitka segmenata, tj do gubitka paketa, veličina prozora spušta se na 1 MSS, a potvrde čekaju dolazak potrebnog paketa za nastavak transmisije podataka s time da će sljedeći segment imati broj najvećeg mogućeg segmenta koji dotada nije već prebačen.



Na slici vidimo, iako malu, promjenu veličine okvira u trenutku TCP retransmission paketa gdje se veličina prozora spustila na 1 MSS (502 okteta).